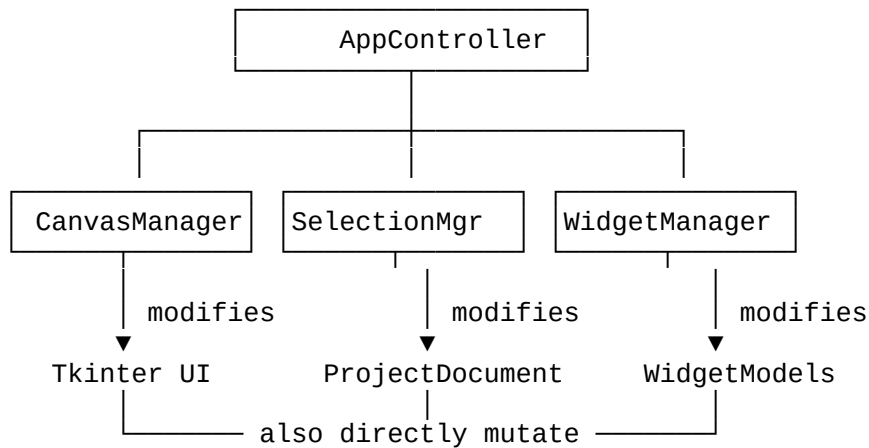


1. Current Structure (Simplified)



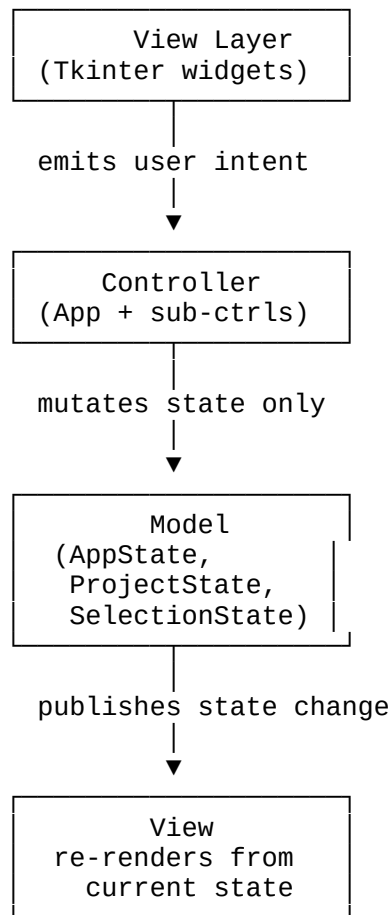
Problems in this shape

- Managers both:
 - Bind UI events
 - Mutate model
 - Update view
- State is modified in multiple places
- Cross-calls between managers likely exist
- Undo/redo must intercept many mutation points

This is why scaling feels harder over time.

2. Target MVC Architecture

Now the same system with strict separation.



How Data Flows

Dragging a widget:

1. View detects mouse drag
2. View emits: `MoveWidgetIntent(widget_id, dx, dy)`
3. Controller receives intent
4. Controller calls: `AppState.move_widget(...)`
5. Model updates canonical state
6. Model emits `StateChanged`
7. View re-renders from state

No manager directly mutates widget data.

No view directly edits model fields.

Current Architecture

Current Component	MVC Role After Refactor
<code>_dataclasses</code>	Model
<code>ProjectDocument</code>	Model
<code>AppController</code>	Root Controller
<code>CanvasManager</code>	Split → <code>CanvasView</code> + <code>CanvasController</code>
<code>SelectionManager</code>	Split → <code>SelectionView</code> + <code>SelectionController</code>
<code>_commands</code>	Controller-side state transition wrappers

Optional: Event Bus Layer

A bus, that sits between view and controller:

View → EventBus → Controller → Model → EventBus → View

That prevents direct cross-calls entirely.

Core Principle

Current system:

Many components mutate shared state.

MVC system:

Only controllers mutate state. Views only render.

High-Level Strategy

Gradually constraining mutation paths until MVC naturally emerges.

PHASE 0 — Stabilization

0.1 Safety Nets

Before structural changes:

- Add minimal tests for:
 - Project serialization/deserialization
 - Widget creation
 - Widget movement
 - Undo/redo basic flow
 - Manually test:
 - Dragging
 - Resizing
 - Selection
 - Save/load
-

PHASE 1 — Explicit State Changes

Goal: Centralize state without changing flow.

1.1 Introduce AppState

Create:

```
class AppState:
    def __init__(self):
        self.project = ProjectDocument(...)
        self.selection = SelectionState(...)
        self.mode = DesignerMode(...)
```

Do NOT move logic yet.

Just create a single container.

1.2 Route All State Access Through AppState

Instead of:

```
self.project_document
self.selection_manager.selected
```

Access:

```
self.state.project
self.state.selection
```

Still allow mutation — just funnel access through one object.

This step is mechanical and safe.

PHASE 2 — Centralize Mutation Points

2.1 Identify All Places Where Model State Changes

Search for:

- `.x +=`
- `.y +=`
- `.append(`
- `.remove(`
- `.selected =`
- Any model attribute mutation

List them.

2.2 Introduce Explicit State Methods

Instead of:

```
widget.x += dx
```

Introduce:

```
state.move_widget(widget_id, dx, dy)
```

Move mutation logic into the state layer.

Managers still call it — but no direct mutation anymore.

Goal:

All model mutations happen in one file.

This is a massive milestone.

PHASE 3 — Decouple View From State Mutation

3.1 Prevent Views From Modifying State

In:

- CanvasManager
- SelectionManager
- WidgetManager

Replace direct mutation with calls to controller/state methods.

If a manager binds a Tkinter event:

Before:

```
on_drag():  
    widget.x += dx
```

After:

```
on_drag():  
    controller.handle_drag(widget_id, dx, dy)
```

Controller:

```
def handle_drag(...):  
    state.move_widget(...)
```

Now mutation is centralized.

PHASE 4 — Make Rendering State-Driven

4.1 Introduce `render()` Methods in Views

CanvasView:

```
def render(state):  
    for widget in state.project.widgets:  
        canvas.coords(...)
```

Now, instead of manually moving canvas items everywhere:

After state mutation:

```
state.move_widget(...)  
canvas_view.render(state)
```

This eliminates:

- UI drift from state
 - hidden positional inconsistencies
-

PHASE 5 — Introduce State Change Notification (Optional but Powerful)

Once mutation is centralized:

Add:

```
class AppState:
    def __init__(...):
        self._listeners = []
    def subscribe(self, fn):
        self._listeners.append(fn)
    def _notify(self):
        for fn in self._listeners:
            fn(self)
```

After any mutation:

```
self._notify()
```

Now views subscribe to state changes.

PHASE 6 — Split Managers Into View vs Controller

CanvasManager becomes:

- CanvasView
- CanvasController

View:

- Tkinter only
- Emits intent

Controller:

- Calls state mutation
 - Coordinates undo/redo
-

PHASE 7 — Refactor Undo/Redo to Wrap State Mutations

Command objects should wrap:

```
state.move_widget(...)
```

Instead of calling widget and state separately.

Undo should only reverse state mutations.

PHASE 8 — Remove Cross-Manager Calls

Search for:

```
self.other_manager.do_something()
```

Replace with:

- Controller calls
- Or event dispatch

Managers should not talk to each other directly.

Only through:

- Controller
 - Or event system
-

PHASE 9 — Audit for Hidden State

Common hidden state:

- Canvas item IDs
- Selection flags inside widgets
- Temporary drag offsets
- UI-only flags

Ensure:

- Domain state lives in model
 - UI ephemeral state lives in view only
 - They are not mixed
-

PHASE 10 — After All This, Introduce Strict MVC Folder Structure

Only when architecture is clean:

```
app/  
  model/  
  view/  
  controller/
```

Do not start here.

Earn this structure gradually.
